

BELGIUM CAMPUS ITVERSITY
SEN381

BUGBUSTERS

CIVICCONNECT PROJECT TEAM

MILESTONE 1

Engineering Foundation & Requirements Baseline

PED v1.0 — Engineering Baseline

TEAM MEMBERS

CHRISTIAN JANSE VAN RENSBURG (601840)

MARCO REINERS (578056)

WALDO BLOM (578068)

PROJECT DOCUMENT DETAILS

COURSE Software Engineering 381 (SEN381)

PROJECT CivicConnect

SUBMISSION DATE [9 September 2026]

Controlled engineering documentation • Team BugBusters

Table of Contents

Problem, Stakeholder Analysis & Business Value.....	3
Problem and business need	3
Stakeholder analysis.....	3
Scope Baseline & Constraints.....	5
Scope Baseline.....	5
In Scope	5
Out of Scope	7
Deferred Scope.....	7
Constraints	8
Requirements & Acceptance Criteria.....	10
Functional Requirements	10
Non-Functional Requirements	12
RTM & Traceability Foundation.....	13
Initial RTM.....	13
Requester RTM.....	13
Staff RTM	13
Management RTM	14
Engineering Decision Log	14
Risk & Forward Engineering Considerations	20
Initial Risk Register	20
Forward Engineering Considerations.....	23
Security, Authentication and Access Control	23
Data Persistence, Integrity, Auditability and Recovery	23
Testability, Verification and CI Quality Gates	23
Deployment, Storage, Free-Tier Limits and Operational Cost	24
Responsive UI and Stakeholder-Specific Mobile Use	24
GitHub Governance	24
Branches.....	24
Pull requests (PR's) and Code review process.....	25
Issues.....	26
Milestones.....	26
Labels.....	26
References.....	27
Baseline Sign off.....	27

Problem, Stakeholder Analysis & Business Value

Problem and business need

The problem is that the community focused organization currently manages service requests through different communication platforms: email, telephone calls, WhatsApp messages, spreadsheets and paper-based protocols. This approach creates operational problems such as: requests are frequently duplicated, overlooked, or lost between channels; requesters don't have visibility into the status of their requests; staff struggle to coordinate and prioritise tasks; management does not have enough information about the outstanding, overdue and resolved work; there is no way to keep track of the different requests logged.

The organisation requires a digital platform that can provide a reliable, and easy way to submit, manage, monitor, and report on these different requests. This digital platform aims to solve these operational problems identified above. Stakeholders each have different business needs will interact with the application in different ways, this is further explained in the next section.

Stakeholder analysis

The project consists of 5 different identified stakeholders, each of them having different needs and ways in which they will interact with the digital platform. Some stakeholders might have conflicts or competing expectations. Below is the breakdown for each stakeholder:

1. Product owner

This is the person who wants the system in the first place. They are the “community-focused organization” that identified the problem. Their responsibility is to ensure that they clearly communicate their needs and expectations for the system that needs to be developed.

They do not submit actual issues. They review the system during the milestones to ensure that it meets the expectations and aligns with the business needs and functional requirements.

Conflict / competing expectation: The Product owner wants the digital platform to have as much functionality as possible within the fixed budget and schedule. This could clash with Development teams need for a stable scope and enough time to build a quality product that is also maintainable. If the scope is not controlled properly, this could lead to scope creep. That is why scope will be controlled and any changes in scope will not just be accepted, they will first be considered against the baseline scope identified and only in the event that scope change is deemed absolutely necessary, it will be accepted and be properly documented.

2. Development team (Waldo, Christian and Marco – “Bugbusters”)

This is us (Waldo, Christian and Marco) the “Bugbusters” team who are actually developing the project based on the identified requirements. Our responsibility is to ensure that we clearly understand the business need in order to identify the requirements of the project. But also, to ensure that we build a system that is maintainable, reliable and secure.

We will test the system to ensure that it works correctly. We will not add additional items to the scope before properly considering the implications of doing so, also in the event that the scope changes we will properly document the decision and the change.

Conflict / competing expectation: We must balance the speed at which we develop the application with the quality and maintainability of the end product. With a fixed schedule and free tier

constraints we must choose quality and sustainability over just implementing something that works and gets the job done.

3. Requester

This is the person who experiences the actual problem i.e. a fault, security concern, lost property, or any other operational request.

Their responsibility is to provide accurate and clear information about the issue they are experiencing, and to label the issue correctly according to the request category. They should also respond to staff when asked for more information about their issue.

Their core need is to stay updated on their request. They want as much visibility as possible when it comes to their logged request, i.e. knowing when it was received, assigned, in progress and resolved. They want a system that is easy to use and easy to interact with, they don't want something is complex and difficult to use.

Conflict / competing expectation: Requesters want an experience that is simple and quick however the staff and management need sufficient information to be able to resolve the issue that the requester is experiencing. These needs clash and need to be balanced, though the digital platform that will be developed.

4. Management

Note: Each department manager (e.g., IT Manager, Facilities Manager) is not a separate stakeholder; Management overall is treated as a single stakeholder group.

Management oversees their specific category (i.e. IT sees IT related requests, Facilities Manager sees the requests associated with facility requests) of requests and monitors activity through a dashboard (see FR-301 of the functional requirements section), which thereby provides the visibility of active and closed requests. In order to improve operational efficiency and reduce management workload, staff may self-assign tasks. However, Management still has the final authority to approve, override, or redirect requests as needed.

Their responsibility is to ensure that the issues assigned to their department is resolved within the expected time frame. Also, to authorize release of contact information of the Requesters to the staff only when needed, for example to get further context about an issue when it was not clearly stated (Scope item nr. 9 & out of scope item 2).

Conflict / competing expectation: Management want detailed reporting and accountability of their staff (who did what, when etc.). However, staff want minimal data entry and an interface that is easy and allows for fast data entry. Privacy of the requesters must also be ensured, information must only be provided when absolutely necessary as the requesters value their privacy of their data but at the same time if they are not specific enough about their issue, they will need to be contacted for further clarification.

5. Staff

Note: Each staff member is not a separate stakeholder; The staff overall is treated as a single stakeholder group.

The staff are the people who are actually assigned to investigate, handle and resolve the service requests made by the requester.

Their primary responsibility is to accept and claim ownership of the issue they are assigned to, including status updates, resolution details and contacting the requester in the event there is clarification needed (Scope item nr. 9 & out of scope item 2).

Their core need is to clearly see the issues that they are assigned to, and to be able to update the status of the requests they are working on. They want an interface that is easy to interact with and that requires minimal clicks and fast loading time (As they are going to be using this system a lot).

Conflict / competing expectation: Staff may resist to use this newly proposed solution in the event that it is slower than the current informal methods (WhatsApp, email, telephone calls etc.). In order to ensure that they will want to use the system, we must make the system as easy as possible to use, while still ensuring management has detailed reporting and accountability of their staff (who did what, when etc.).

Scope Baseline & Constraints

Scope Baseline

In Scope

These are the core features and things that our application will and must be able to do. These are the things that we are promising to ensure that our system will perform, this is the baseline scope and will be adjusted as the project progresses (note in the event that the scope is adjusted we will properly document these changes):

1. User login, registration and authentication
Requesters will be able to register and log in to access account specific functionality. They will also be able to update their personal information and settings such as if they want receive email push notifications.
This authentication will also help with staff and management functionality as they will also have accounts. This will help with things such as staff being assigned to a specific department and also to a specific management person.
2. Requesters submit a new service request
Requesters will be able to submit a new service request containing all the required information.
3. Add images to a request
Requesters will also be able to attach images to a specific request in the event that it is needed. However, images will be limited to a certain file size (still to be determined at a later stage what this established limitation is, as it depends on the database architecture limitations and free tier availability).
4. Controlled request categorization
Requesters will select a request category from a list of available categories. These categories will then be used to make the request available to the relevant department. For example, IT related issues will only go to the IT department and NOT across the all the departments. This will help to better keep track of what the issue pertains to.
5. Status tracking of a request
The requesters will be able to view the current status of their submitted request as it changes.

Staff will be able to change the status of only the issues they are assigned to. Management then also has the dashboard view of the different requests and their status within their department and which staff member is assigned to the specific issue

6. View request history
Requesters will be able to view the history of their previously submitted requests. Managers will also be able to view all the past requests logged within their department. Staff will be able to view their assigned request history.
7. Feedback via email push notification.
As an additional feature that we will implement is the ability for requesters to receive email notifications as their status of their task changes. This feature will be able to be toggled on / off. This is also why we require a user registration with email as we will use this email address to send the push notifications to.
8. Search and filter requests
Staff and Management will be able to search, filter and sort available service requests using relevant criteria.
9. View requester details
Authorized staff will be permitted access to contact details associated with a requester. This is how the staff member will be able to get further clarification on an issue if required. The manager must first approve access to this information for the staff member.
10. Assign / accept responsibility
Authorized staff will be able to accept responsibility for the appropriate unassigned requests. Meaning that they will be able to assign themselves to a specific request only within their department.
11. Management assign and reassignment
Although staff are able to assign themselves to a task within their department, management still has the ability to assign or reassign service requests to the appropriate staff member within their department. This thereby allows managers to only assign task when needed, which allows them to spend more time on other responsibilities they have to attend to such as monitor the performance of their staff via the dashboard.
12. Update status
The assigned staff member of a specific task will have the ability to update the status of the issue as it progresses.
13. Resolve and close request
The assigned staff member will be able to resolve and close the request. Management is able to revert this closer if deemed the issue is not yet resolved.
14. Management dashboard and reporting
Management will be able to view information to open, overdue, resolved and closed requests. Management will be able to analyse request information using the relevant search and filter tools available.
15. Digital platform Mobile accessibility

For our proposed digital platform, some of the screens we will adapt for mobile as this will improve the user experience for the requesters.

Out of Scope

These are explicit decisions that that we will not be moving forward with and are excluded from the baseline scope.

1. AI chatbot
We will not create an AI chatbot for users to communicate with.
It is unlikely that users will make use of this feature. Also, when implementing something like this it can be complex there are certain risks such as people misusing the chatbot for personal reasons (Schuman, 2026). There are security and implementation risks associated that are not worth the potential value that it could add (Schuman, 2026).
2. Internal real time messaging system
Requesters won't have direct communication with staff via the system. Staff will send an email or call the user only when the issue on the request is not clearly stated. The system will have the option to update the status of the issues.
3. Video file upload
Requesters will not be able to attach video files to the service requests. Uploads will be restricted to images only due to storage limitations (images will take less space to store).
4. Voice input
Communication is limited to text only. Dealing with different audio files can get complex and also require a more data storage capacity. This potential feature is not worth the additional storage capacity.
5. Social media login
We will provide one method of registration and that is email along with a password. We will not be integrating Google login, Login via Facebook. This overcomplicates the issue and requires an additional API's that could change or break. This could add to the technical debt of the application.

Deferred Scope

These are recognised possible future things we could implement in future milestones, but we deliberately postpone these issues due to schedule, cost, risk or missing evidence.

1. Auto assign issues to the same task
Ideally, it would be beneficial for the system to automatically group multiple requests to the same underlying issue. For example, a specific website that the community focused organization manages and is in charge of goes down. They might get multiple requests that the website is no longer accessible. So instead of dealing with each individual request we treat it as single issue. This could be advantageous, but this decision is deferred as we would first like to implement the basic functionality first. This is a "nice to have".
2. Portal for requesters to review their interactions with the staff
Adding a reviews portal where requesters could review their interaction with a specific staff member if they had one. This is also another "nice to have".

3. Additional role for across all management departments

An additional user role that will have access to a dashboard to view overall metrics of the system such as how many tasks assigned to what department, how long on average does each department take to resolve specific issues. It would give metrics about the performance of the staff and management overall at a glance (Like an Admin user). So, this is not department specific but a user that has a greater level of access. We defer this decision as we would first like to establish a baseline system and then if there is available time for this feature, we will implement it

4. Extent of the Mobile adaptability

For our proposed digital platform, we would like to have a mobile adaptability for the requesters. But we don't yet know if we want to apply this same mobile adaptability for the Managers or Staff. Further investigation into their needs is required and also a deeper understanding of what needs to be displayed on the screens is required. For example, the management dashboard will most likely not be made responsive on a mobile screen as this is something that won't be easily achievable.

Constraints

Constraint	Description	Engineering Implication
Team Size and Resources	The project must be completed by a team of only 3 members.	Because development resources are limited, we must avoid complex architectures that require more overhead. If one team member falls behind, the other two will do the workload, which means code must be readable and documented.
Schedule	The project must progress through 4 formal milestones within 7 weeks.	Time is heavily limited, so scope must be flexible. If a chosen technology is too difficult to learn or if development falls behind the schedule, then we must drop the "could have" and "should have" features rather than to miss a milestone's deadline.
Cost	The team must use free services where possible and identify operational limits.	This heavily restricts our tech stack for M2. We cannot use premium services. We must actively monitor our usage of free limits to ensure we do not hit a paywall.
Scope control	Committed scope must be baselined, uncontrolled scope creep is not acceptable.	Once M1 document is signed off, any new feature ideas cannot be silently added. It must go through a formal Change Request and impact analysis to ensure it does not interfere with our schedule or quality.
Evidence	Attributes must be defined and later supported by measurable evidence and not just claims.	We cannot just simply say the code works; we must prove it. This is implied we must allocate time to write automated test and configure continuous integration pipeline to act as a quality gate before any code merges into the main branch.
Security	Security must be treated as a lifecycle wide engineering responsibility not just a final add-on.	We must implement strict config management from day one. Things like database passwords and API keys must never be committed to our GitHub. This does require us to establish a secure environment strategy.
Technology and team capability	No technology stack is prescribed but decisions must be justified against the	We cannot pick a popular framework just because it looks trendy. If we pick a tech stack that nobody in the team knows there is a learning curve risk and that will threaten our schedule.

	constraints and our team capability.	
--	---	--

Requirements & Acceptance Criteria

Functional Requirements

Priority:

M – Must have.

S – Should have.

C – Could have.

How the Requirement IDs work; The first number is for stakeholder when it comes to FR and category when it comes to NFRs. The second and 3rd number is just for numbering the requirement.

Requirement ID	Group of Feature	Description	Priority	Source/Stakeholder	Acceptance Criteria
FR-101	Submission	The system must allow users to submit a service request using a category mechanism.	M	Requester	When valid details are submitted when the user clicks, save/submit then a unique ID is generated.
FR-102	Tracking	The system must allow users to view the status of requests.	M	Requester	Given that the user inputs their unique ID, then the ticket status is displayed.
FR-103	History	The system should allow authenticated users to view a history of their requests	S	Requester	Given that the user is logged in, when navigating to their profile, then a list of their past request should render.
FR-104	Feedback	The system must provide feedback when a user's request is updated.	M	Requester	When a request is accepted, rejected, updated or completed, the system must generate an appropriate notification for the requester.
FR-105	Authentication & Registration	The system must allow requesters to register an account and authenticate before accessing account-specific functionality.	M	Requester	Given valid registration information, the system must create a requester account. Given valid registered credentials, the system must authenticate the requester. Invalid credentials must be rejected.
FR-106	Attachments	The system should allow requesters to attach supported image files to a service request.	S	Requester	Given a requester creating or submitting a request, when a supported image is attached, the image must be associated with the correct service request and be retrievable by an authorised user.
FR-201	Assignment	The system must allow staff to accept responsibility for a ticket.	M	Staff	Given that an unassigned ticket and the staff user clicks assign, then they must become the record owner.
FR-202	Workflow	The system must allow staff to update request statuses through controlled transitions.	M	Staff	Given a ticket, if updated to a different status then the status state is saved in the database.

FR-203	Staff View	The system must allow the staff to search, filter and sort request using criteria.	M	Staff	Given the staff dashboard, when filtering by criteria, then only tickets related to that criteria are displayed.
FR-204	Resolution	The system must allow staff to record actions, comments or resolution information.	M	Staff	Given that a ticket is being resolved, then the staff member must input a resolution comment before closing it.
FR-205	Request Details	The system must allow authorised staff to view the full permitted details of service requests relevant to them.	M	Staff	Given an authorised staff member opening an available request, the system must display the permitted request information associated with that request.
FR-206	Request Closure	The system must allow authorised staff to resolve and close a service request through the controlled workflow.	M	Staff	Given an authorised staff member resolving a request, when all required resolution information has been provided and a permitted closing transition is performed, the request must be recorded as resolved or closed.
FR-207	Department-Relevant Access	The system must make service requests available to authorised staff according to the request category and relevant staff responsibility or department.	M	Staff	Given a service request assigned to a defined category, authorised staff within the relevant responsibility area must be able to access it, while staff outside the authorised scope must not be able to access it.
FR-301	Dashboard	The system must provide a dashboard to identify open, overdue and closed requests.	M	Management	Given that a manager accesses the dashboard, then a visual metric of active vs closed tickets should render.
FR-302	Auditing	The system should support reporting on request lifecycle history for accountability.	S	Management	Given a completed ticket, then a timestamp audit log of all transitions should be viewable.
FR-303	Assignment & Reassignment	The system must allow authorised management to assign or reassign responsibility for a service request to an appropriate staff member.	M	Management	Given an eligible service request and authorised manager, when a valid staff member is selected and the assignment is confirmed, that staff member must become the recorded assignee and the assignment change must be recorded.
FR-304	Request Analysis	The system must allow authorised management to analyse service-request information using relevant dimensions such as category and status.	M	Management	Given an authorised manager viewing service-request information, when a supported category, status or other approved dimension is selected, the resulting information must correspond to the selected criterion.
FR-305	Contact Information Authorisation	The system must restrict staff access to requester contact information and allow authorised management to control access where additional clarification is required.	M	Management	Requester contact information must not be made available to an unauthorised staff member. Where the required access has been authorised for a request, the permitted information must become available only to the appropriate authorised staff member.

Non-Functional Requirements

Requirement ID	Category	Description	Measurable Acceptance criteria	Justification
NFR-101	Security	The system must enforce authentication and data protection.	Passwords must be hashed. Data in transit must use https. Role-Based Access Control must prevent requesters from accessing staff endpoints. (measurable through test gates, binary pass/fail)	"Security is a lifecycle wide responsibility not a final add-on", Because CivicConnect handles sensitive community data such as locations and contact details, failing to secure trust early creates privacy risks.
NFR-201	Performance	The platform must respond efficiently to user inputs to prevent duplicate submissions.	95% of all standard API read and write requests must compile in less than 800 milliseconds under normal system load.	If the system responds slow, then users will click "submit/save" multiple times, leading to duplicated tickets. By setting a millisecond limit, gives the team a measurable goal.
NFR-301	Constraints	The architecture must operate for free (open source) or within low cost.	The production environment must run on free-tier services without exceeding monthly quotas.	Cost is an engineering constraint in this Project.
NFR-401	Compatibility	Given that the decision is made that the system needs to operate on multiple different platforms -The public portal must be usable on modern desktop and mobile browsers. Please note this decision will only be made in milestone 2.	The UI must pass responsive visual testing on the newest versions of Chrome, Safari, Edge and Firefox without horizontal scrolling on mobile.	Since native mobile applications are out of scope, the web interface is the only way citizens can submit a new service request. Requesters will most likely report issues on the go, using their mobile browsers, making responsiveness a crucial attribute.
NFR-501	Reliability	The system must handle erroneous inputs without crashing.	100% of the form submissions must have backend validation. The system must return human readable http error codes instead of stack traces.	If the system crashes silently on bad input, tickets are lost. Validation ensures data integrity and provides feedback to the user.

RTM & Traceability Foundation

Initial RTM

According to the CivicConnect Master Project Brief, the team must maintain one evolving Requirements Traceability Matrix (RTM), with requirements linked from their stakeholder/source through later design, implementation, testing and acceptance/release evidence. The Milestone 1 brief requires the initial RTM to establish traceability from the stakeholder/source to the requirement and acceptance criteria, while providing a structure that can be expanded with later lifecycle evidence.

In this document, the source/stakeholder, requirement description, priority and acceptance criteria are already defined in the Functional and Non-Functional Requirements tables. The RTM therefore uses the Requirement ID as the link to these definitions and extends the traceability structure to future design/architecture, GitHub Issue/Pull Request, implementation, verification/testing and acceptance/release evidence. Fields relating to future milestones are marked as pending until the relevant controlled evidence is produced.

Requester RTM

Requirement ID	Design / Architecture	Issue / PR	Implementation Evidence	Verification / Test	Acceptance / Release Evidence
FR-101	Pending - M2	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-102	Pending - M2	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-103	Pending - M2	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-104	Pending - M2	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-105	Pending - M2	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-106	Pending - M2	Pending - M3	Pending - M3	Pending - M3	Pending - M4

Staff RTM

Requirement ID	Design / Architecture	Issue / PR	Implementation Evidence	Verification / Test	Acceptance / Release Evidence
FR-201	Pending - M2	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-202	Pending - M2	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-203	Pending - M2	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-204	Pending - M2	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-205	Pending - M2	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-206	Pending - M2	Pending - M3	Pending - M3	Pending - M3	Pending - M4

Management RTM

Requirement ID	Design / Architecture	Issue / PR	Implementation Evidence	Verification / Test	Acceptance / Release Evidence
FR-301	Pending - M2	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-302	Pending - M2	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-303	Pending - M2	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-304	Pending - M2	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-305	Pending - M2	Pending - M3	Pending - M3	Pending - M3	Pending - M4

Engineering Decision Log

The Engineering Decision Log records significant decisions made during Milestone 1 so that their rationale, trade-offs, risks and later consequences remain traceable throughout the project. Where insufficient evidence currently exists to support a responsible decision, the decision is deliberately deferred and the information required before commitment is identified. In accordance with the CivicConnect Master Project Brief, significant decisions are recorded together with their context, constraints, alternatives, rationale, risks, supporting evidence and later consequences.

Field	Entry
Decision ID	DEC-001
Description	Requirement Identification Convention
Context	CivicConnect requirements must remain identifiable and traceable across the PED, RTM and later engineering artefacts. The team therefore needs a consistent naming and numbering convention.
Constraints	Requirements must use unique and stable identifiers and must remain traceable throughout later design, implementation, testing and change activities.
Alternatives	1. Use a single sequential numbering scheme for all requirements. 2. Separate Functional and Non-Functional Requirements and use numbering groups to indicate stakeholder or quality category.
Decision	Functional Requirements will continue to use stakeholder-based numbering, with FR-1xx for Requester requirements, FR-2xx for Staff requirements and FR-3xx for Management requirements. Non-Functional Requirements will use category-based numbering.
Rationale	The numbering structure allows the team to identify the primary stakeholder or quality category associated with a requirement while maintaining stable identifiers for traceability. As the scope now contains related behaviour across Requesters, Staff and Management, this separation also reduces ambiguity when requirements are later linked to design, implementation, testing and change evidence.
Trade-offs	The convention improves organisation and traceability but requires the team to maintain the numbering structure consistently as new requirements are added or changed.
Risks	1. Incorrect, duplicated or inconsistently referenced IDs could break links between the requirements tables, RTM, GitHub issues, later tests and release evidence.
Evidence	Functional Requirements table, Non-Functional Requirements table and Initial RTM within PED
Later Consequence	The same requirement IDs will be used in later architecture/design artefacts, GitHub issues and PRs, implementation evidence, tests and acceptance evidence.

Field	Entry
Decision ID	DEC-002
Description	GitHub Branching Strategy
Context	The team requires a controlled GitHub workflow that prevents uncontrolled changes to the project's stable code and documentation while allowing three team members to work concurrently.
Constraints	The main branch must be protected, substantive changes must enter through Pull Requests, and changes require two approvals from team members other than the author.
Alternatives	1. Use only a protected main branch with temporary feature branches. 2. Use protected main and dev branches with temporary feature/documentation/fix branches.
Decision	The team will use main and dev as the primary controlled branches, with temporary feature branches created for individual pieces of work. Branch prefixes such as feature/ and docs/ will be used to indicate the purpose of the branch.
Rationale	Separating active development from the controlled main branch provides an additional integration point while still allowing individual work to occur independently and be reviewed before it enters a controlled branch.
Trade-offs	The additional dev branch creates more branch-management overhead and requires the team to keep merges and branch states controlled.
Risks	1. Poor branch management could cause merge conflicts, divergence between dev and main, or confusion about which branch represents the current controlled state. 2. Because the scope contains functionality that cuts across user roles and departments, parallel work on related features could create integration conflicts if branches remain isolated for too long.
Evidence	GitHub repository configuration, branch rulesets, documented branch naming convention, branch history and Pull Request evidence.
Later Consequence	Future implementation, testing and integration work will follow this branching structure, so GitHub history can provide traceable evidence of controlled engineering changes.

Field	Entry
Decision ID	DEC-003
Description	Exclusion of High-Complexity Communication and Media Features
Context	CivicConnect Project could support additional interaction mechanisms such as an AI chatbot, internal real-time messaging, video uploads and voice input.
Constraints	The project is limited by schedule, team size, cost, storage capacity, security considerations and the requirement to prioritise the core service-request workflow.
Alternatives	1. Include all proposed communication/media features. 2. Include selected additional features. 3. Exclude these features and focus on the core request workflow.
Decision	AI chatbot functionality, internal real-time messaging, video uploads and voice input will be excluded from the committed baseline. CivicConnect will use structured request information, supported image attachments and external communication such as email or telephone when further clarification is required
Rationale	These additional features are not necessary to satisfy the core CivicConnect business need and would introduce additional implementation, storage, security, testing and maintenance complexity.
Trade-offs	Users receive fewer communication/input options, but the reduced scope allows the team to focus available time and resources on the core request lifecycle, security, traceability and quality.

Risks	Some users may prefer richer communication mechanisms. However, including them could increase schedule pressure, storage requirements, integration complexity and technical debt.
Evidence	PED Out-of-Scope section
Later Consequence	Architecture, storage and interface design do not need to support video, audio, chatbot or internal real-time messaging capabilities. Adding any of these later would require formal change analysis.

Field	Entry
Decision ID	DEC-004
Description	Defer Final Technology Stack and Architecture Selection
Context	CivicConnect now has a more defined baseline that includes account authentication, departmental request routing, image attachments, management dashboards, email notifications, role-based access and responsive requester-facing interfaces. These requirements create significant implications for persistence, security, deployment, file storage and interface design.
Constraints	Technology selection must consider requirement fit, team capability, learning curve, cost, security, testing support, development-environment compatibility, maintainability and deployment compatibility.
Alternatives	1. Select a preferred stack and architecture immediately during M1. 2. Make a provisional choice based primarily on existing team familiarity. 3. Deliberately defer final selection until M2, when sufficient requirements and comparison evidence are available.
Decision	Deferred to Milestone 2. The team will not commit to a final technology stack or software architecture during M1.
Rationale	Selecting a stack before the requirements, quality attributes and constraints are sufficiently baselined could prematurely constrain the solution and create avoidable technical, schedule, cost or deployment problems.
Trade-offs	Deferring preserves flexibility and enables evidence-based evaluation, but technology evaluation must be completed early enough in M2 to avoid reducing implementation time.
Risks	Deciding too early could result in an unsuitable technology choice. Deferring for too long could create schedule pressure during M2 and M3.
Evidence	1. M1 scope/constraints, Requirements and Acceptance Criteria, Forward Engineering Considerations, team-capability assessment and later M2 technology comparison evidence. 2. Future evidence list will include: • support for authentication and role/department-based access • image storage/file-size limitations • email notification support • responsive web interface support • database and persistence requirements • deployment/free-tier compatibility • security support • team familiarity and learning curve • test and CI compatibility
Later Consequence	M2 must evaluate viable technology and architecture alternatives against the baselined requirements, NFRs and constraints before committing to implementation.

Field	Entry
Decision ID	DEC-005
Description	Defer CI / GitHub Actions Configuration
Context	The final system will require automated build, testing and quality controls, potentially implemented through GitHub Actions. However, the exact CI workflow depends on the technology stack, build process and testing tools selected later.
Constraints	M1 establishes the engineering foundation rather than implementing the CI pipeline. The final stack and architecture have not yet been selected.
Alternatives	1. Configure GitHub Actions immediately using assumed technologies. 2. Create only a placeholder workflow. 3. Defer the exact CI implementation until the build, dependency and testing requirements are known.
Decision	Deferred. The team will use CI and automated quality controls later in the project, but the exact GitHub Actions workflow will not be selected or implemented during M1.
Rationale	A meaningful CI pipeline must reflect the actual technology stack, dependency installation process, build commands and automated tests. Configuring it before those are known could create throwaway work or unnecessarily constrain later choices.
Trade-offs	Deferring avoids premature configuration but means CI setup will become an important early activity once the stack is selected.
Risks	If the CI decision is deferred too long, automated build and testing evidence may be delayed. If configured too early, the workflow may require significant rework
Evidence	PED decision/deferment note; later technology decision; GitHub Actions workflow and pipeline evidence in subsequent milestones.
Later Consequence	Once the M2 technology stack is selected, the team must define the appropriate build, test and quality checks and configure the CI workflow accordingly.

Field	Entry
Decision ID	DEC-006
Description	Use Email-Based Registration Rather Than Social Login
Context	CivicConnect requires user accounts for Requesters, Staff and Management. Account identity is also needed to associate requests, department membership, management responsibility, requester details and email notifications with specific users.
Constraints	The project has limited development time and resources. Additional external authentication providers would introduce third-party API dependencies, configuration, security considerations and additional testing requirements.
Alternatives	1. Email/password registration managed by CivicConnect. 2. Third-party/social login such as Google or Facebook. 3. Support both approaches.
Decision	CivicConnect will use an email-based registration and authentication mechanism. Third-party/social login will not form part of the committed baseline.
Rationale	Email is already required for requester notifications and provides a single controlled identity mechanism without introducing additional external authentication integrations.

Trade-offs	The team must manage the security responsibilities associated with authentication and password storage, while users do not receive the convenience of signing in through an existing external account.
Risks	Poor authentication or password-management implementation could expose user accounts. The selected implementation will therefore need appropriate security controls and verification.
Evidence	PED In-Scope and Out-of-Scope sections; Authentication Functional Requirement; Security NFR.
Later Consequence	M2 architecture and security design must provide an appropriate authentication mechanism, secure credential handling and role/account association. External identity-provider APIs do not need to be supported.

Field	Entry
Decision ID	DEC-007
Description	Department-Based Request Access and Management-Controlled Contact Information
Context	CivicConnect must ensure that requests are handled by relevant departments while limiting unnecessary access to request and requester information. Staff may need requester contact details when additional clarification is required.
Constraints	Requests may contain sensitive contact and location information. Staff should only access requests relevant to their department, and access to requester contact information should be controlled.
Alternatives	1. Allow all Staff to view and claim all requests. 2. Restrict requests by department but allow staff unrestricted access to requester contact information. 3. Restrict requests by department and require management approval before Staff gain access to requester contact information.
Decision	Service requests will be categorised and made available only to the relevant department. Staff may accept responsibility only for eligible requests within their department. Requester contact information will remain restricted until authorised by the relevant manager where additional clarification is required.
Rationale	Department-based access reduces irrelevant request exposure and supports clear operational ownership. Management-controlled access to contact information limits unnecessary exposure of requester data while still allowing Staff to obtain clarification when justified.
Trade-offs	The approach introduces additional permissions, approval logic and management involvement compared with allowing unrestricted access. It may also slightly delay Staff access to requester contact information.
Risks	Incorrect category assignment could make a request unavailable to the correct department. Poorly implemented access controls could either block legitimate Staff or expose requester information to unauthorised users.
Evidence	PED In-Scope section; Stakeholder Analysis; FR-207 Department-Relevant Access; FR-305 Contact Information Authorisation; NFR-101 Security.
Later Consequence	M2 architecture and data design must support user roles, department relationships, request-category mapping, authorisation rules and controlled access to requester information. M3 testing must verify both permitted and denied access scenarios.

Field	Entry
Decision ID	DEC-008
Description	Partial Mobile Responsiveness Scope
Context	CivicConnect will be used by different stakeholder groups with different interface needs. Requesters may need to submit and track service requests while away from a desktop environment, while Staff and Management interfaces are likely to contain denser operational information such as assignment controls, filtering tools and management dashboards.
Constraints	The project has a limited development schedule and a three-person team. Making every interface fully responsive would increase design, implementation and testing effort. The exact mobile usability needs of Staff and Management have also not yet been sufficiently established.
Alternatives	1. Make all Requester, Staff and Management interfaces fully responsive from the outset. 2. Provide no specific mobile adaptation and target desktop use only. 3. Prioritise mobile responsiveness for Requester-facing interfaces and defer the extent of mobile adaptation for Staff and Management interfaces until their needs and screen complexity are better understood.
Decision	Requester-facing functionality will be designed to support mobile browser use. The extent of mobile responsiveness for Staff and Management interfaces is deliberately deferred until further analysis of their operational needs and interface requirements has been completed.
Rationale	Requesters are more likely to need convenient access when submitting, tracking or reviewing requests from different locations. Staff and Management interfaces may require larger or information-dense screens and committing to full responsiveness before the relevant layouts and workflows are understood could introduce unnecessary development and testing effort.
Trade-offs	Prioritising Requester mobile usability reduces immediate responsive-design effort and allows the team to focus on the most likely mobile-use case. However, Staff and Management may have a less optimised mobile experience if later analysis does not justify additional responsive design.
Risks	If Staff or Management later require significant mobile access, additional redesign and testing may be necessary. Conversely, attempting to make all interfaces fully responsive too early could increase schedule pressure and reduce time available for higher-priority functionality and quality controls.
Evidence	PED In-Scope section under Digital Platform Mobile Accessibility; Deferred Scope section under Extent of Mobile Adaptability; NFR-401 Compatibility; later M2 wireframes/prototypes and stakeholder/interface analysis.
Later Consequence	M2 user-interface design must prioritise responsive behaviour for Requester-facing screens. Staff and Management mobile requirements must be reviewed before their interface designs are finalised. NFR-401 may need to be refined so its measurable acceptance criteria clearly state which stakeholder interfaces and screen sizes are required to support responsive use.

Please note that due to the space constraints in Microsoft Word the format in which the Engineering Decision Log is displayed in this PED is not the same as the table format we will record and work with in the Engineering_Decision_Log.md file in our GitHub repository.

Risk & Forward Engineering Considerations

Initial Risk Register

In accordance with the CivicConnect Master Project Brief, the Risk Register is maintained as a live engineering artefact and records project-specific risks together with their cause, probability, impact, priority, mitigation, contingency, owner and current status. Risks are identified from the current CivicConnect scope, requirements, constraints and engineering decisions and will be reviewed and updated as new evidence becomes available throughout subsequent milestones. Where a risk occurs, it will be treated as an issue and linked to the relevant engineering actions or decisions.

Risk ID	Risk / Description	Cause	Probability	Impact	Priority	Mitigation	Contingency	Owner	Status
RSK-001	Schedule overrun due to scope and learning curve The team may fail to complete the committed CivicConnect scope within the required milestone schedule.	The project contains multiple user roles, authentication, departmental access, dashboards, notifications, image uploads and responsive requester-facing functionality, while the team consists of only three members and the final technology stack has not yet been selected.	Medium	High	Critical	Maintain the baselined scope; prioritise Must-have requirements; evaluate technology choices against team capability and learning curve; track milestone progress through GitHub issues; avoid implementing deferred or out-of-scope features without formal change analysis.	If schedule variance becomes significant, defer or remove lower priority Should/Could requirements through controlled change rather than reducing essential testing, security or quality controls.	Development Team Product Owner	Open - Active
RSK-002	Uncontrolled scope growth Additional feature requests or ideas may expand the committed project scope and threaten delivery of the baseline requirements.	The Product Owner may want additional functionality, and the team has already identified several deferred features that could appear valuable later.	Medium	High	High	Baseline the approved M1 scope; clearly distinguish in-scope, out-of-scope and deferred functionality; require formal change requests and impact analysis before accepting additional features.	Reject or defer proposed changes where the schedule, cost, quality or security impact cannot be accommodated. If a change is necessary, update the requirements, RTM, Risk Register and other affected artefacts through controlled change.	Product Owner Development Team	Open - Active
RSK-003	Unauthorised access to requester information Requester contact information or service-request data may be exposed to users who are not authorised to access it.	CivicConnect requires multiple user roles, departmental restrictions and management-controlled access to requester contact details, creating complex authorisation rules.	Medium	High	Critical	Define clear authentication and authorisation requirements; enforce role and department-based access; restrict contact information by default; include permitted and denied access scenarios in later security testing; prevent secrets from being committed to GitHub.	If an access-control flaw is discovered, restrict or disable affected functionality, correct the authorisation logic, investigate exposed information where applicable and verify the fix before restoring access.	Development Team Management	Open - Active

Risk ID	Risk / Description	Cause	Probability	Impact	Priority	Mitigation	Contingency	Owner	Status
RSK-004	Incorrect request categorisation or departmental routing A service request may be categorised incorrectly and therefore become unavailable to the department responsible for resolving it.	Requesters select the category when creating a request, and that category determines which department can access the request.	Medium	High	High	Use a controlled category list with clear labels/descriptions, ensure management can redirect or reassign requests where necessary, test category-to-department access rules.	Provide Management with the ability to correct or redirect incorrectly categorised requests and ensure the correction is recorded through the audit trail.	Requester Management Development Team	Open - Active
RSK-005	Unsuitable technology or deployment platform selection The technology stack or deployment platform selected during M2 may not adequately support the project's requirements and constraints.	No technology stack is prescribed, while the solution must support authentication, role/department access, image storage, email notifications, responsive interfaces, persistence, testing and deployment within free/low-cost limits.	Low	High	High	Maintain DEC-004 deferment until sufficient evidence exists, compare viable technologies against requirements, team capability, security, deployment compatibility, testing support, storage limitations and cost/free-tier constraints.	If the selected technology proves unsuitable during early M2 validation, revisit the decision before substantial implementation and select the next suitable evaluated alternative.	Development Team	Open - Monitoring
RSK-006	Free-tier resource or service limitations CivicConnect may exceed the limits of free-tier hosting, database, storage or supporting services required by the project.	The project must prefer free or low-cost services, while features such as image storage, database persistence, email notifications and deployment consume platform resources.	Medium	Medium /High	High	Investigate platform quotas before technology commitment, estimate likely storage and service usage, restrict image file sizes, monitor free-tier consumption, prefer technologies that remain practical within the project's expected workload.	Reduce non-essential resource consumption, change service configuration, limit supported upload size or move to another evaluated free/low-cost alternative where feasible.	Development Team Product Owner	Open - Monitoring
RSK-007	Image upload creates storage or security problems User-uploaded images may consume excessive storage or introduce unsafe or unsupported file content into the system.	Image uploads are in scope, but the maximum file size, storage architecture and exact validation approach have not yet been selected.	Medium	Medium /High	High	Define supported image formats and maximum file size after storage/platform evidence is available, validate uploads, restrict unsupported files, consider storage quotas and authorisation when designing retrieval.	Temporarily reduce allowed file size/types or disable affected upload behaviour if storage/security constraints cannot be satisfied until the issue is corrected.	Development Team Requester	Open - Decision Pending M2
RSK-008	Email notification dependency or delivery failure Requesters may not receive status notifications reliably	Email notification delivery will depend on the selected email mechanism/service, configuration and	Medium	Medium	Medium	Select and test an appropriate notification mechanism, allow users to toggle email notifications, ensure	If email delivery fails, retain the correct status within CivicConnect and allow the requester to view it through the	Development Team Requester	Open - Monitoring

Risk ID	Risk / Description	Cause	Probability	Impact	Priority	Mitigation	Contingency	Owner	Status
	even though the request itself is updated correctly.	potentially an external service or provider.				notification failure does not prevent the underlying request update.	platform while the notification mechanism is repaired.		
RSK-09	Team-member availability or knowledge concentration Progress may be delayed if important project knowledge or work becomes concentrated in one team member who becomes unavailable or falls behind.	The project is completed by only three team members and different members may become primary contributors to specific technical or documentation areas.	Medium	High	High	Maintain readable documentation and code, require peer review, ensure all members understand major requirements, decisions and artefacts, use GitHub issues/PRs to keep work visible, avoid creating areas understood by only one team member.	Reassign affected work using the documented artefacts, GitHub history and peer-reviewed contributions; reduce lower-priority work if necessary to protect milestone delivery.	Development Team	Open - Active
RSK-010	Loss or corruption of platform data. Including but not limited to: Service-request records, assignment information, status history or audit information may be lost, corrupted or become inconsistent.	CivicConnect will maintain persistent request lifecycle information, but the database architecture, integrity controls, backup and recovery approach have not yet been selected.	Low/Medium	High	High	Preserve data integrity requirements during M2 design, evaluate persistence technology and transaction/integrity support, define appropriate validation, backup and recovery considerations before production deployment.	Restore from available backup/recovery evidence where possible, correct data-integrity defects and prevent release until critical request/audit information can be stored reliably.	Development Team Management	Open - Monitoring

Forward Engineering Considerations

The considerations below identify why each concern is relevant to the current engineering baseline, which later decisions or activities it may influence, what information is still required before a responsible decision can be made, and the potential consequence of ignoring the concern.

Security, Authentication and Access Control

Security and access control already influence the CivicConnect baseline because the system will store requester information and provide different levels of access to Requesters, Staff and Management. Requests are also restricted according to department, while requester contact information may only be released to Staff after Management approval. This means that later architecture and implementation decisions must support controlled access rather than treating all authenticated users in the same way. The current requirements already establish department-based access and management-controlled contact information.

- **Later decision/activity influenced:** Authentication architecture, role-based access control, department-based permissions, database relationships, API access rules, security testing and deployment configuration.
- **Information still missing:** Final authentication technology, detailed role and permission structure, session or token mechanism, password-storage approach and the technical method used to represent Management approval of contact-information access.
- **Risk if ignored:** Security may be introduced too late in the design process, which could require significant redesign or result in unauthorised users gaining access to requests or requester information

Data Persistence, Integrity, Auditability and Recovery

Reliable data storage is very important when it comes to the CivicConnect project because currently the operational processes suffer from requests being lost, duplicated, overlooked or poorly tracked. The new system must therefore preserve request information, assignments, status changes, resolution details and lifecycle history reliably.

- **Later decision/activity influenced:** Database technology, schema design, relationships, transaction handling, audit logging, backup and recovery planning.
- **Information still missing:** Final database technology, persistence model, audit fields, retention requirements, backup frequency and recovery approach.
- **Risk if ignored:** Poor data design could recreate the original business problem by allowing request records or accountability information to become incomplete, inconsistent or unrecoverable.

Testability, Verification and CI Quality Gates

The requirements and acceptance criteria defined during M1 will determine whether the team can later prove that CivicConnect works correctly. The current PED already requires measurable evidence, automated testing and Continuous Integration rather than relying only on claims that the code works.

- **Later decision/activity influenced:** Test strategy, unit/integration/system testing, testing framework selection, GitHub Actions configuration, merge quality gates and release evidence.
- **Information still missing:** Final technology stack, testing tools, build process, CI workflow, performance-test conditions and quality thresholds.
- **Risk if ignored:** Requirements may be implemented without reliable verification evidence, while vague acceptance criteria could become difficult to test later.

Deployment, Storage, Free-Tier Limits and Operational Cost

CivicConnect must operate within free or low-cost constraints, but the final hosting, database and storage environment has not yet been selected. Features such as image uploads, persistent data, email notifications and authentication may all consume limited platform resources. The PED already identifies free-tier limits as an engineering constraint.

- **Later decision/activity influenced:** Hosting platform, database service, image-storage approach, email service, deployment pipeline and operational-cost decisions.
- **Information still missing:** Storage quotas, database limits, bandwidth limits, image-size limits, expected usage volumes, email-service restrictions and final deployment platform.
- **Risk if ignored:** The selected solution may work during development but exceed free-tier limits, create unexpected costs or require late redesign of storage and deployment components.

Responsive UI and Stakeholder-Specific Mobile Use

Requester-facing mobile accessibility is already part of the baseline, while the extent of mobile responsiveness for Staff and Management has deliberately been deferred. This means the team must support requester mobile use without prematurely committing to making every interface fully responsive.

- **Later decision/activity influenced:** Wireframes, responsive layouts, browser support, UI architecture, usability testing and refinement of NFR-401.
- **Information still missing:** Staff and Management mobile-use requirements, final dashboard complexity, target screen sizes and supported browser/version range.
- **Risk if ignored:** The team could either overcommit to full mobile responsiveness and increase development effort unnecessarily or create requester-facing screens that are difficult to use on mobile devices.

GitHub Governance

Link to our repo: https://github.com/Waldo-Blom/SEN381_Bugbusters

GitHub is a tool but just because you make use of GitHub does not mean that you applied proper software engineering. In order to apply software engineering you need to apply software engineering practices. The following below describes exactly how we will use GitHub throughout the project to ensure these best practices.

Branches

We will have two main branches that we will use `main` and `dev`. From these two branches we will branch out further into subbranches referred to as feature branches.

According to our project brief the main branch should be treated as a fully protected branch that does not allow commits on the branch unless through a pull request and detailed review. We achieve this through applying a ruleset to the `main` and `dev` branch with the following settings:

- **Requires a pull request before merging:** All commits must be made to a non-protected branch and submitted via a pull request before they can be merged into main. Direct development on main it is therefore not possible.
- **Requires approvals:** Pull requests targeting main require a minimum number of approvals. We have set the required number of approvals before merging to 2 thereby matching the project master brief.
- **Force pushes are blocked:** The "Allow force pushes" setting is disabled, so force pushes to main are not permitted.

- **Deletions are blocked:** When the "Allow deletions" setting is disabled, the main and dev branch can't be deleted (Just to avoid accidental deletion of our production and development branches)
- **Do not allow bypassing the above settings:** This setting just makes sure no matter the role that the above-mentioned settings can be bypassed by admin privileges or roles

Let's take a practical example:

Waldo was assigned the following task to complete: "Documentation for "Problem and business need"". Now he has to go and make a change on the `problem-and-business-need.md` file (stored in the docs folder). Instead of just working on the branch `main` (which is against the project spec and not good software engineering practices (as you would never develop straight on the live version of company software)), he instead creates an entire new branch called `docs/waldo-m1-artifacts`. He then in his own local environment makes changes to that mentioned .md file, either through single or multiple commits (whatever fits his use case).

Note: We use naming prefixes (like docs/ and feature/) to categorize the type of work. GitHub utilizes these prefixes by allowing us to filter branches by name, making it easy to see all documentation branches or all feature branches at once, this helps to improve navigation and organization in the repository (Github, 2025).

Here are other examples of feature branches from our naming convention:

- docs/ai-usage-register-m1-waldo
- feature/NFR-improve-login-loading-time
- feature/FR-implement-login

As the project evolves, we might add additional branching prefixes, we will identify where certain changes align and agree on a prefix for that code change type. When a new one is added we will also update it in the `Github-governance.md` file, under the docs folder of our repo.

Note branches get deleted after each milestone, this is done to keep the repo clean and structured, so we don't end up with for example 70 feature branches that are no longer used. But before deleting a feature branch it is first merged with the relevant branch either `main` or `dev` depending on the scenario and then that branch absorbs the feature branch commits and changes through the PR, nothing is lost and the commit history is kept, that branch just can be accessed anymore but this is fine.

See our current branches here: https://github.com/Waldo-Blom/SEN381_Bugbusters/branches

Pull requests (PR's) and Code review process

Marco and Christian then first have to review this change within the pull request, treating the Pull request (PR) as a place to do a code review, meaning they look at Waldo's proposed changes and then comments and gives feedback. After both Marco and Christian approve this change made by Waldo (Waldo can't approve his own changes, he only verifies that his work is indeed correct before opening a pull request). This applies for all members, the scenario above is just for demonstrative purposes.

As per our Ruleset, the main branch requires at least 2 approvals (enforced automatically). We do not allow self-approval on any branch that requires reviews. Any merge conflicts are then resolved (if there are any).

Example PR: https://github.com/Waldo-Blom/SEN381_Bugbusters/pull/2

Issues

When referring to issues in GitHub we don't necessarily mean it in terms of a software bug (Github, 2025). This refers to a specific task that has to be completed (Github, 2025). For our project we start by identifying what the deliverables are for a specific milestone and then assign we distribute the workload and assign specific tasks to specific people. But just because one person is in charge of a specific part of the workload does not mean that person is the only one working on that, we look at what that person has done and then review it and incorporate our feedback.

Example issue: https://github.com/Waldo-Blom/SEN381_Bugbusters/issues/3

Milestones

The milestones on our GitHub page relate to the actual milestones of this project i.e. Milestone 1 on GitHub consists of the issues that are related to Milestone 1 project deliverables.

Every issue must be assigned to only one milestone (M1, M2, M3, or M4). Assigning issues to milestones allows us to track our overall progress, the milestone view automatically shows what percentage is complete (based on closed vs. open issues) and helps us see if we are on track or falling behind.

Link Milestones: https://github.com/Waldo-Blom/SEN381_Bugbusters/milestones

Labels

We use labels in our PR's and issues just to mark issues more clearly, for example:

- By priority:
 - `priority: Urgent` – Blocking issues, critical bugs etc
 - `priority: High` - important but not a blocking issue
 - `priority: Medium` - normal priority
 - `priority: Low` - Nice to have, can defer
- By the task type:
 - `Feature` - New functionality added
 - `Documentation` - Improvements or additions to documentation
 - `Bug` - Something isn't working
 - `Enhancement` - Improve something / change an existing behaviour
- By the type of work:
 - `Front end` - UI / Client side changes
 - `Back end` - Server / API work
 - `Database` - Data layer / queries

In the future as the project evolves, we might add additional labels as we see fit in order to provide extra clarity where necessary. Or we might remove some that are not used.

Link to the labels: https://github.com/Waldo-Blom/SEN381_Bugbusters/labels

References

Github. “About Issues - GitHub Docs.” *GitHub Docs*, 2025, <https://docs.github.com/en/issues/tracking-your-work-with-issues/learning-about-issues/about-issues>. Accessed 7 Sept. 2026.

Github. “Viewing Branches in Your Repository - GitHub Docs.” *GitHub Docs*, 2026, <https://docs-internal.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-branches-in-your-repository/viewing-branches-in-your-repository>. Accessed 7 Sept. 2026.

Schuman, Evan. “AI Token Freeloaders Are Coming for Your Customer Support Chatbot.” *CIO*, 8 Apr. 2026, <https://www.cio.com/article/4155404/ai-token-freeloaders-are-coming-for-your-customer-support-chatbot.html>. Accessed 7 Sept. 2026.

Baseline Sign off

Appendix D — Baseline Sign-Off Template

Project	CivicConnect
Baseline Type	PED Milestone 1
Version	Version1
Date	9 September 2026
Scope reviewed	☒ YES ☐ NO
Requirements/traceability checked	☒ YES ☐ NO
Risk review completed	☒ YES ☐ NO
Repository/governance controls checked	☒ YES ☐ NO
Outcome	☒ ACCEPTED ☐ CONDITIONALLY ACCEPTED / REVISION REQUIRED